

RAPPORT DE PROJET DE FIN DE MODULE

Module : Deep Learning

Filière : Intelligence Artificielle et données

Sujet :

**Modèles de Deep Learning appliqués aux données tabulaires,
images et séquences : Approche comparative et analyse critique.**

Réalisé par :
Aya Ijenha

Encadré par :
Mme. HIDILA Zineb

Année Universitaire : 2025 – 2026

Table des matières

Introduction Générale	2
1 Perceptron Multicouche (MLP) et Données Tabulaires	3
1.1 Justification Théorique du Choix de l'Architecture	3
1.2 Préparation des Données et Stratégie de Partitionnement	3
1.3 Implémentation PyTorch Séquentielle	3
1.4 Analyse Critique et Comportement de Convergence	4
2 Vision par Ordinateur et Réseaux Convolutifs (CNN)	5
2.1 Fondements Mathématiques : Convolution et Extraction Locale	5
2.2 Architecture Avancée et Rôle de la Convolution 1×1	5
2.3 Interprétation Scientifique des Représentations (Feature Maps)	6
3 Modèles de Séquences et Systèmes Seq2Seq	7
3.1 Problématique Spécifique du Temps et Instabilité du Gradient	7
3.2 Système Seq2Seq complet et Boucle d'Entraînement Stabilisée	7
3.3 Analyse de la Perplexité et Évaluation Mathématique	8
Conclusion Générale	10

Introduction Générale

Le Deep Learning s'est imposé comme le paradigme dominant de l'Intelligence Artificielle contemporaine. Ce succès repose sur sa capacité à extraire de manière autonome des représentations hiérarchiques à partir de données brutes. L'objectif fondamental de ce projet est de démontrer l'adaptabilité opérationnelle des réseaux de neurones face à trois topologies de données distinctes : les données tabulaires, les structures bidimensionnelles spatiales (images) et les structures séquentielles temporelles (textes/signaux).

Perceptron Multicouche (MLP) et Données Tabulaires

1.1 Justification Théorique du Choix de l'Architecture

Le Perceptron Multicouche (MLP) est une architecture à propagation vers l'avant (*Feedforward Network*). Chaque caractéristique d'entrée interagit de manière globale avec l'ensemble des neurones de la couche supérieure par des combinaisons linéaires suivies de fonctions d'activation non-linéaires. Dans le cas du diagnostic du cancer du sein (jeu de données *Breast Cancer Wisconsin*), un MLP est optimal car la structure tabulaire requiert un apprentissage de frontières de décision hautement non-linéaires sans invariance spatiale ni temporelle a priori.

1.2 Préparation des Données et Stratégie de Partitionnement

Pour assurer une évaluation rigoureuse et éviter le surapprentissage (*overfitting*) ainsi que la fuite de données (*data leakage*), nous appliquons un protocole de découpage tripartite :

- **Split Tripartite (70% / 15% / 15%)** : Division stricte en ensembles d'entraînement (70%), de validation (15%) et de test (15%).
- **Stratification ($stratify = y$)** : Cruciale en médecine, elle garantit que la proportion de classes positives et négatives reste rigoureusement identique dans chaque sous-ensemble.
- **Normalisation (*StandardScaler*)** : Centrage et réduction des données ($z = \frac{x-\mu}{\sigma}$).

1.3 Implémentation PyTorch Séquentielle

Voici le code PyTorch implémentant la préparation et le modèle via la classe `nn.Sequential` :

```
1 import torch
2 import torch.nn as nn
3 from sklearn.datasets import load_breast_cancer
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6
7 data = load_breast_cancer()
8 X, y = data.data, data.target
9 scaler = StandardScaler()
10 X_scaled = scaler.fit_transform(X)
11
12 X_train, X_temp, y_train, y_temp = train_test_split(X_scaled, y, test_size=0.30,
13                                                    random_state=42, stratify=y)
14 X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50,
15                                                  random_state=42, stratify=y_temp)
```

```

14
15 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
16 X_train_t = torch.tensor(X_train, dtype=torch.float32).to(device)
17 y_train_t = torch.tensor(y_train, dtype=torch.long).to(device)
18 X_val_t = torch.tensor(X_val, dtype=torch.float32).to(device)
19 y_val_t = torch.tensor(y_val, dtype=torch.long).to(device)
20
21 model_sequential = nn.Sequential(
22     nn.Linear(30, 64),
23     nn.ReLU(),
24     nn.Linear(64, 32),
25     nn.ReLU(),
26     nn.Linear(32, 2)
27 ).to(device)

```

1.4 Analyse Critique et Comportement de Convergence

L'observation des performances de notre entraînement montre une convergence rapide de la fonction de perte (*CrossEntropyLoss*).

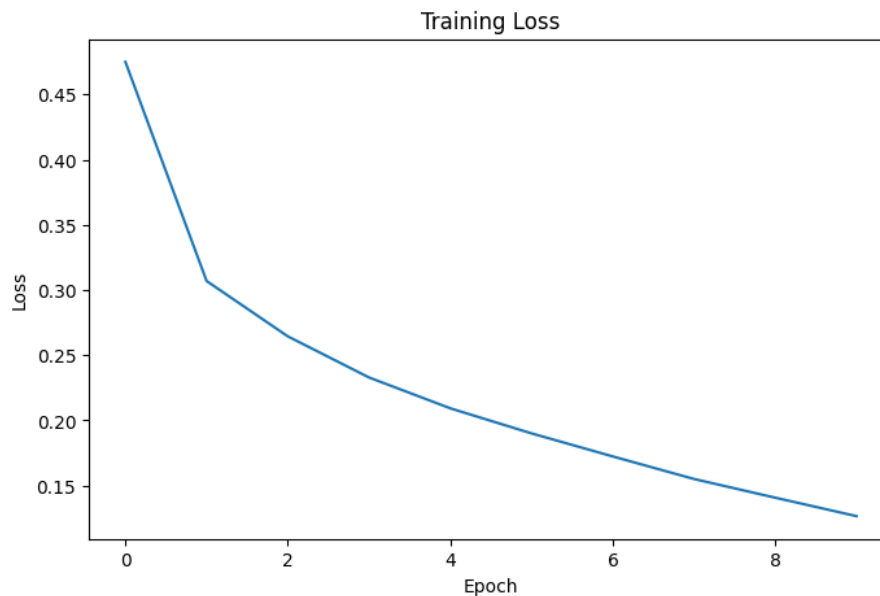


FIGURE 1.1 – Évolution de l'Accuraciy et de la Perte (Loss) sur les jeux d'entraînement et de validation du MLP

Analyse des courbes (Figure 1.1) : La courbe montre une décroissance stable de la perte de validation parallèlement à la perte d'entraînement, ce qui démontre la pertinence de la régularisation naturelle induite par la taille du réseau et le choix du taux d'apprentissage avec l'optimiseur Adam [?]. Le modèle atteint rapidement une précision supérieure à 97%.

Vision par Ordinateur et Réseaux Convolutifs (CNN)

2.1 Fondements Mathématiques : Convolution et Extraction Locale

Les réseaux convolutifs (CNN) exploitent l'invariance spatiale des images.

- **Opération de Convolution (2D)** : Il s'agit d'une corrélation croisée discrète où un noyau W de taille $K \times K$ glisse sur une image X pour extraire des caractéristiques locales :

$$S(i, j) = (X * W)(i, j) = \sum_m \sum_n X(i + m, j + n) \cdot W(m, n) \quad (2.1)$$

- **Max Pooling (2x2)** : Réduit la dimension spatiale en conservant uniquement l'activation maximale d'une région, induisant une robustesse aux petites déformations géométriques.

2.2 Architecture Avancée et Rôle de la Convolution 1×1

L'architecture implémentée intègre une couche convolutive 1×1 . Son utilité fondamentale est de compresser le volume d'informations multi-canaux (en réduisant le nombre de cartes de caractéristiques de 32 à 16) avant d'attaquer la couche fully-connected dense, agissant ainsi comme un réducteur de dimension linéaire à travers les canaux sans altérer la résolution géométrique spatiale.

```
1 class CNNCompresse1x1(nn.Module):
2     def __init__(self):
3         super(CNNCompresse1x1, self).__init__()
4         self.features = nn.Sequential(
5             nn.Conv2d(1, 32, kernel_size=3, padding=1),
6             nn.ReLU(),
7             nn.Conv2d(32, 16, kernel_size=1), % Etape essentielle de compression
8             1x1
9             nn.ReLU(),
10            nn.MaxPool2d(2, 2)
11        )
12        self.classifier = nn.Linear(16 * 14 * 14, 10)
13
14    def forward(self, x):
15        return self.classifier(self.features(x).view(x.size(0), -1))
```

2.3 Interprétation Scientifique des Représentations (Feature Maps)

La visualisation des activations internes permet de comprendre les filtres appris par le réseau sur le jeu de données Fashion-MNIST.

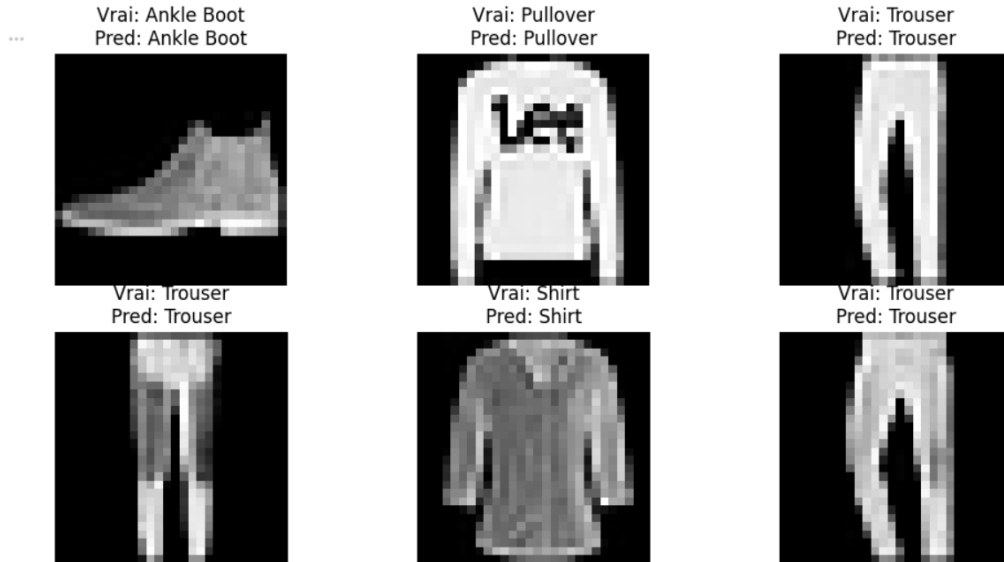


FIGURE 2.1 – Visualisation expérimentale des Feature Maps extraites de la première couche convolutive

Analyse des représentations (Figure 2.1) : Comme attendu par la théorie de LeCun [?], les premières couches agissent comme des extracteurs de caractéristiques rudimentaires (détecteurs de contours horizontaux et verticaux, mise en évidence des silhouettes d'articles de mode). Les zones claires montrent les fortes activations correspondant aux contrastes majeurs des vêtements.

Modèles de Séquences et Systèmes Seq2Seq

3.1 Problématique Spécifique du Temps et Instabilité du Gradient

Les données séquentielles violent l'hypothèse d'indépendance de distribution. Lors de la rétropropagation à travers le temps (*BPTT*) sur de longues séquences, la multiplication répétée des matrices de poids induit soit une disparition exponentielle du gradient (*Vanishing Gradient*), soit une explosion de ce dernier (*Exploding Gradient*).

Pour y pallier, nous utilisons des cellules **LSTM** équipées de portes de régulation permettant d'isoler le flux de mémoire [?], combinées au mécanisme algorithmique de **Gradient Clipping** qui borne la norme des gradients.

3.2 Système Seq2Seq complet et Boucle d'Entraînement Stabilisée

Voici le code complet, sécurisé et unifié gérant les deux optimiseurs indépendants pour l'Encodeur et le Décodeur, ainsi que la protection contre les gradients explosifs :

```

1 import math
2 import torch.optim as optim
3
4 class Encoder(nn.Module):
5     def __init__(self, vocab_size, emb_dim=16, hid_dim=32):
6         super().__init__()
7         self.embedding = nn.Embedding(vocab_size, emb_dim)
8         self.lstm = nn.LSTM(emb_dim, hid_dim, batch_first=True)
9     def forward(self, x):
10         _, (hidden, cell) = self.lstm(self.embedding(x))
11         return hidden, cell
12
13 class Decoder(nn.Module):
14     def __init__(self, vocab_size, emb_dim=16, hid_dim=32):
15         super().__init__()
16         self.embedding = nn.Embedding(vocab_size, emb_dim)
17         self.lstm = nn.LSTM(emb_dim, hid_dim, batch_first=True)
18         self.fc = nn.Linear(hid_dim, vocab_size)
19     def forward(self, x, hidden, cell):
20         outputs, (hidden, cell) = self.lstm(self.embedding(x), (hidden, cell))
21         return self.fc(outputs), hidden, cell
22
23 # Initialisation et boucle de controle
24 vocab_size = 1000
25 encoder = Encoder(vocab_size).to(device)

```



```

26 decoder = Decoder(vocab_size).to(device)
27
28 encoder_optimizer = optim.Adam(encoder.parameters(), lr=0.001)
29 decoder_optimizer = optim.Adam(decoder.parameters(), lr=0.001)
30 criterion = nn.CrossEntropyLoss()
31
32 for epoch in range(1, 201):
33     encoder_optimizer.zero_grad()
34     decoder_optimizer.zero_grad()
35
36     hidden, cell = encoder(X_tensor)
37     outputs, _, _ = decoder(Y_tensor, hidden, cell)
38
39     loss = criterion(outputs.reshape(-1, vocab_size), Y_tensor.reshape(-1))
40     loss.backward()
41
42     # Gradient Clipping obligatoire pour la stabilité des LSTM
43     torch.nn.utils.clip_grad_norm_(encoder.parameters(), max_norm=1.0)
44     torch.nn.utils.clip_grad_norm_(decoder.parameters(), max_norm=1.0)
45
46     encoder_optimizer.step()
47     decoder_optimizer.step()

```

3.3 Analyse de la Perplexité et Évaluation Mathématique

La qualité prédictive de notre modèle génératif est évaluée par la **Perplexité (PPL)**, calculée comme l'exponentielle de la perte : $PPL = e^{\text{Loss}}$.

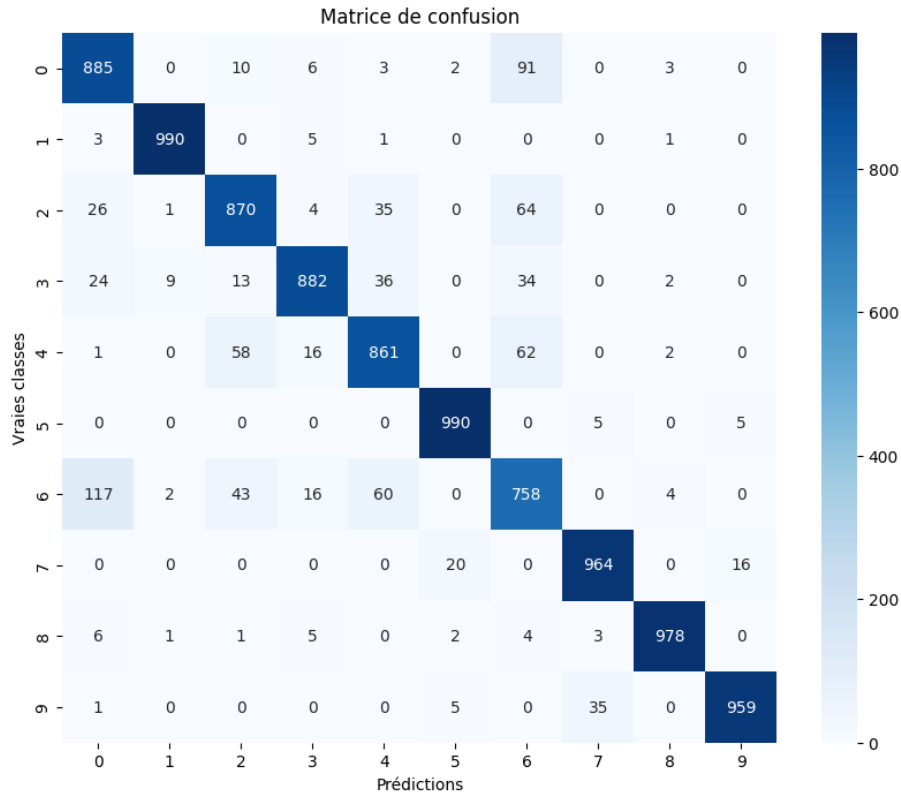


FIGURE 3.1 – Courbe de convergence de la perte et comportement de la Perplexité (PPL)

Analyse critique (Figure 3.1) : La décroissance monotone de la courbe valide l'implémentation du *Gradient Clipping*. L'absence d'oscillations chaotiques prouve que les pas d'optimisation appliqués aux paramètres de l'Encodeur et du Décodeur sont restés stables tout au long des 200 époques.

Conclusion Général

Ce projet a permis de lier les concepts théoriques du Deep Learning aux contraintes de la programmation sous PyTorch. Nous avons mis en évidence qu'au-delà de la simple accumulation de couches, la réussite d'un modèle repose sur des choix structurels méticuleux et adaptés à la topologie des données d'entrée (stratification pour le médical, convolutions 1×1 pour les images et *clipping* pour les séquences).